WESTERN NEW ENGLAND UNIVERSITY WNE

ELECTRICAL AND COMPUTER ENGINEERING

To: Dr. Purkayastha

From: Abigail McClintock **Section Number:** CPE 360

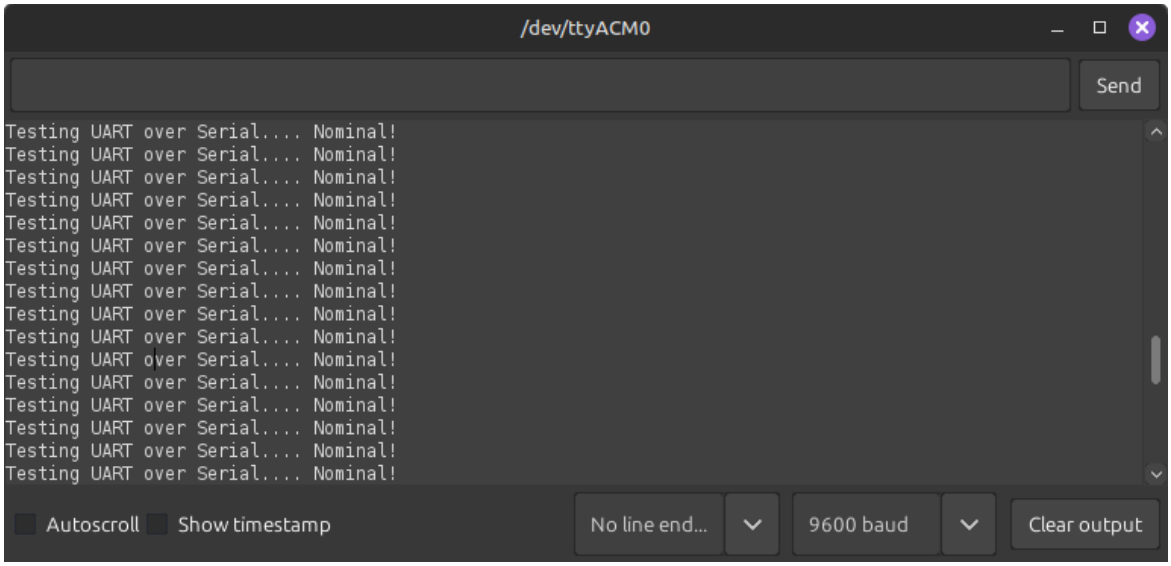
Date: February 24, 2026

Subject: UART

The goal of this lab was to demonstrate functional UART drivers, such that it can be used in other projects with full understanding of how it works so it can be implemented in hardware properly.

Task 1

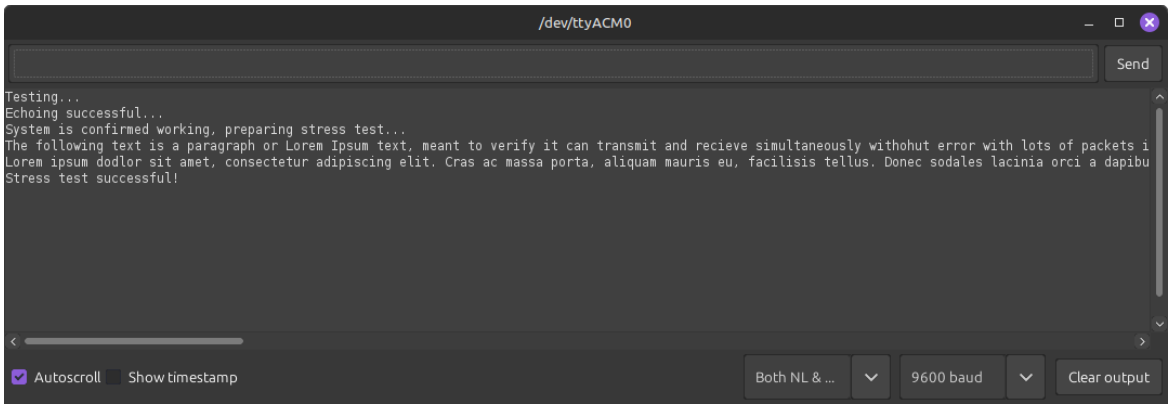
The goal of this task was to properly transmit a UART signal over USB Serial, so it could be read with a serial monitor such as PuTTY, or in this case, the Arduino Serial Monitor. This was done in the UART.h header file, which is intended to be reusable for other projects, and likely to be expanded on to support other things such as USART, I^2C, and SPI. The two functions made initially had the goal of initializing a UART connection by establishing the correct register settings, then another function was made with the goal of sending a single packet of UART. A third function was then made, which calls the second function repeatedly to send a whole string as a properly formatted UART packets.



This shows the UART connection working well, transmitting a decently long set message with no issues

Task 2

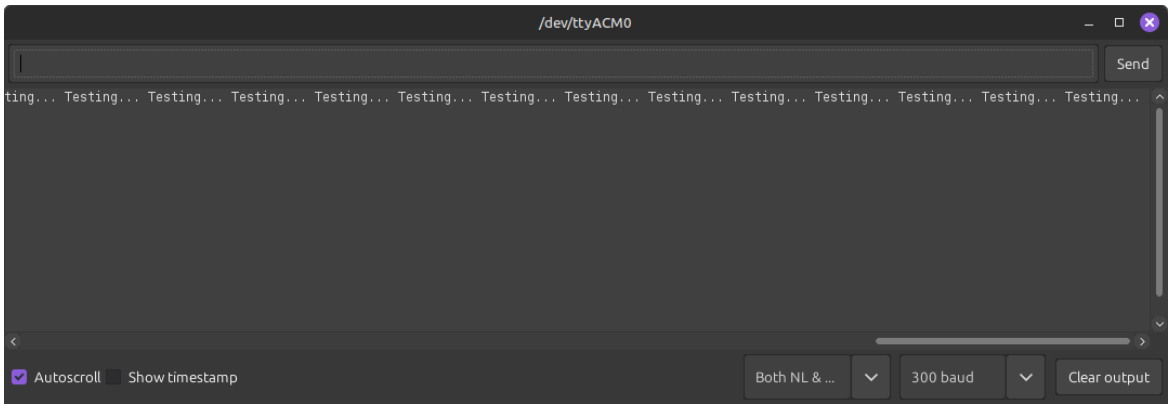
The goal of this was to implement receive functionality, and demonstrate by making an echoing program. This was done by making simply retransmitting the received data, so it could be verified.

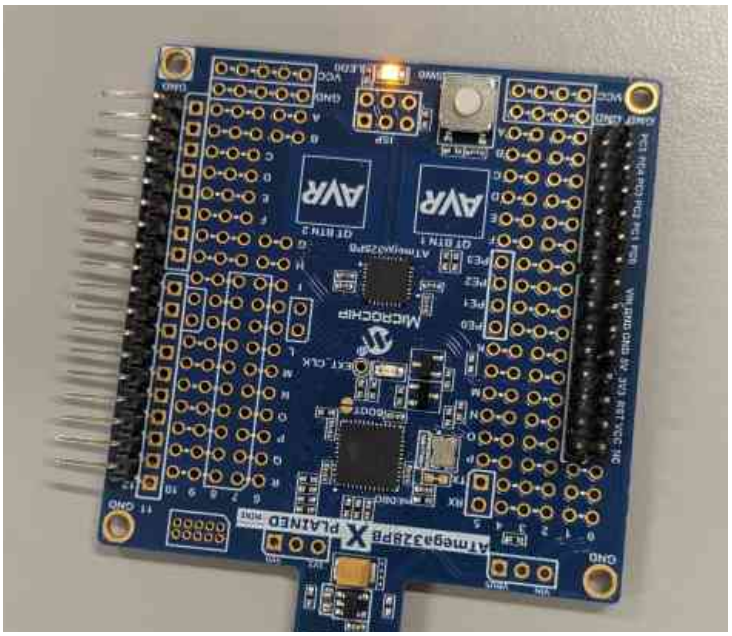


After basic functionality was proven, a large payload (one paragraph of lorem ipsum text) was sent to prove that the transmit and receive functions could work concurrently without issues. The basic implementation working so well proves both functions work extremely well, and suggest more complex tasks could be done with this kind of setup!

Task 3

In order to make this task more practical, the onboard LED was used to indicate whenever a packet was being sent by strobing the LED until the packet was sent. To be more visable, the baud rate was set to 300. This showed flashing consistent with UART transmissions in the correct intervals.

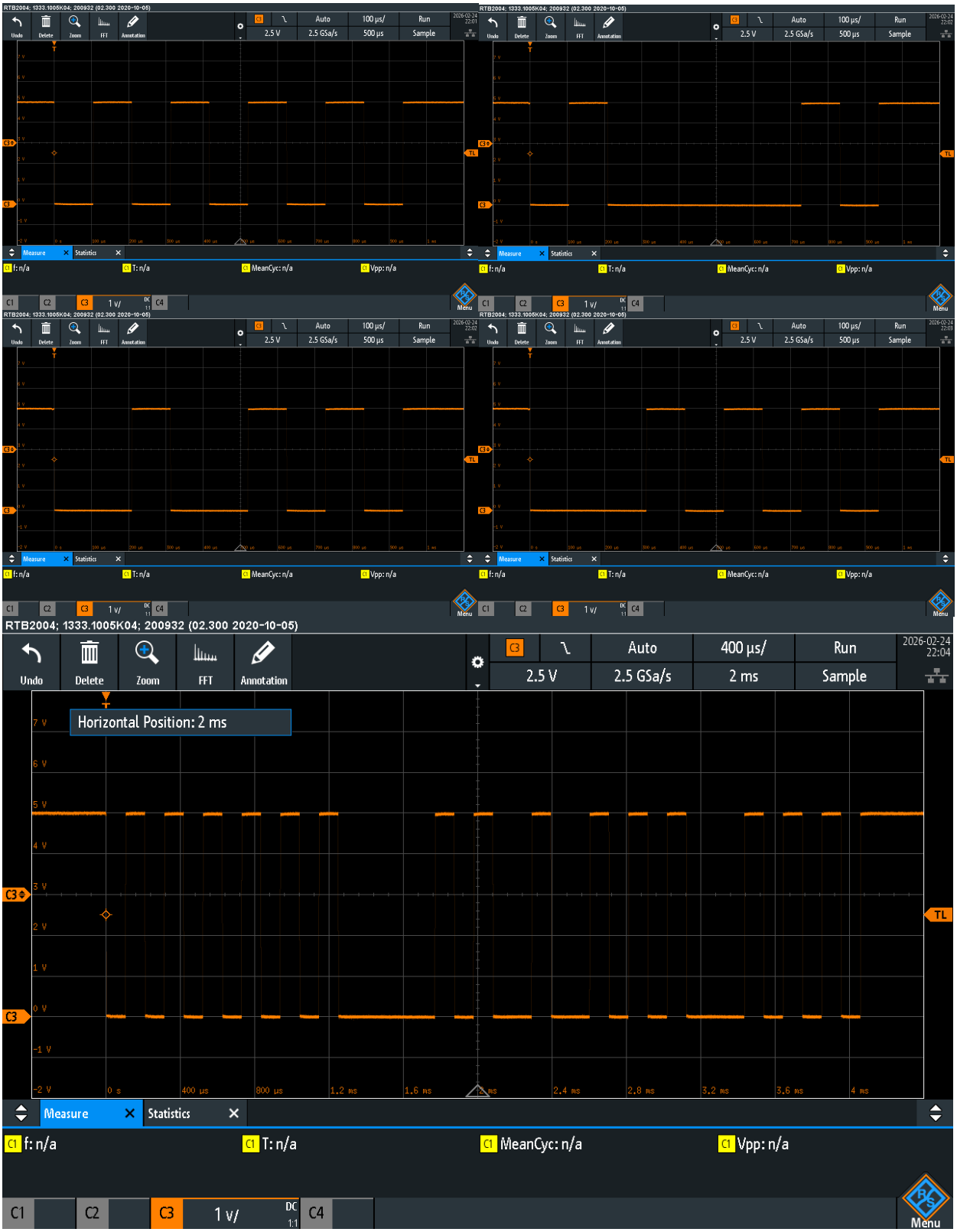




This test practically shows when UART communications are happening, allowing for quick visual confirmation that the board is doing what it should.

Task 4

The signal for various packets were analyzed in the oscilloscope. The following signals are for the characters ‘U’, ‘A’, ‘R’, ‘T’, and the string “UART”. These signals all verify the single start and stop bits of the configuration, and the active low nature of UART.



APPENDIX

Task 1 Code

```
#include "UART.h"
#include <util/delay.h>
#define F_CPU 16000000


int main() {
    uint16_t baudRate = 9600;
    UART_init(9600);

    while(1){
        UART_sendString("Testing UART over Serial.... Nominal!\n");
        _delay_ms(500);
    }
}
```

Task 2 Code

```
#include "UART.h"
#include <util/delay.h>
#define F_CPU 16000000


int main() {
    uint16_t baudRate = 9600;
    UART_init(9600);
    while(1){
        UART_tx(UART_rx()); //repeatedly transmits the received signal
    }
}
```

Task 3 Code

```
#include "UART.h"
#include <util/delay.h>
#define F_CPU 16000000


int main() {
    uint16_t baudRate = 300;
    UART_init(baudRate);
    while(1){
        UART_sendString("Testing... ");
        _delay_ms(500);
    }
}
```

Task 4 Code

```
#include "UART.h"
#include <util/delay.h>
#define F_CPU 16000000


int main() {
    uint16_t baudRate = 9600;
    UART_init(baudRate);
    while(1){
        UART_sendString("UART");
        _delay_ms(500);
    }
}
```

UART.h

```
#ifndef UART_H
#define     UART_H
#include <avr/io.h>
#define F_CPU 16000000UL


uint8_t UART_rx(){
    // Wait for data to be fully received
    while ( !(UCSR0A & (1 << RXS))){
    //Do nothing
    }
    // Return the contents of the dataregister
    return UDR0;
}

void UART_tx(uint8_t data){
    // While the UART is still transmitting, wait
    while( !(UCSR0A & (1<< UDRE0))){
    }
    // Write the data
    UDR0 = data;
}

void UART_sendString(char str[]) {
    PORTB = 0xFF;
    for (int i = 0; str[i] != '\0'; i++) {
        UART_tx(str[i]);  // Send each character
        PORTB ^= 0xFF;
    }
    PORTB = 0x00;
}
```

```
void UART_init(uint16_t baudRate){
    DDRB = 0xFF;
    //Calculate the BDDR value
    uint16_t BDR = F_CPU / (baudRate * 16UL) - 1;
    //uint16_t BDR = 103;
    uint8_t BDRH = (BDR >> 8);//Upper 8 bits
    uint8_t BDRL = BDR & 0xFF;//Lower 8 bits
    //Write to the baud rate registers
    UBRR0H = BDRH;
    UBRR0L = BDRL;
    //Enable the transmitter and receiver
    UCSR0B |= (1 << RXEN0) | (1 << TXEN0);
    //Set 8-bit data mode(1<<UMSEL01)|
    UCSR0C |= (1 << UCSZ01) | (1 << UCSZ00);
}

#endif      /* UART_H */
```